



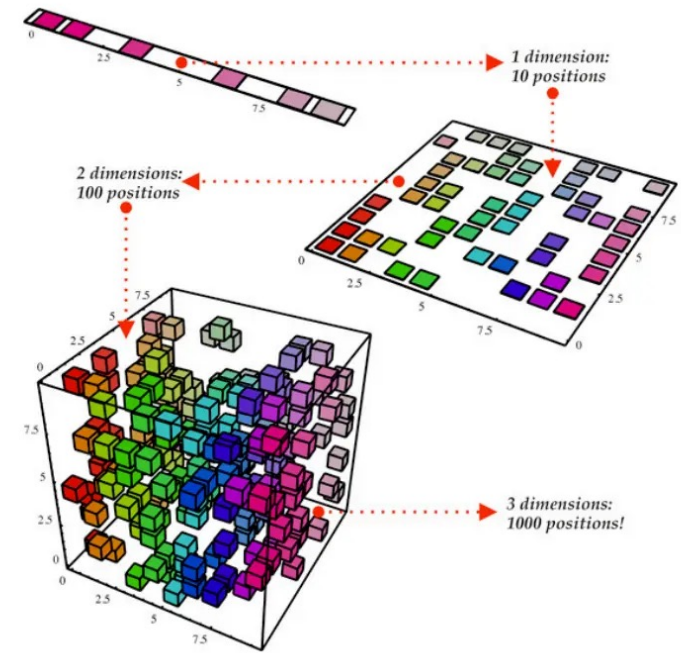
Dimensionality Reduction

Outline

- Motivation: Why Dimensionality Reduction?
- Mathematical Foundations
- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- PCA vs LDA
- t-SNE

Why Dimensionality Reduction?

- High-dimensional data problems:
 - Noise increases with dimensions
 - Distance becomes less meaningful (curse of dimensionality)
 - Hard to visualize (>3D)
- Goal: Reduce dimensions while keeping important information



Two Approaches

- PCA (Principal Component Analysis):
 - unsupervised
 - preserve variance
- LDA (Linear Discriminant Analysis):
 - supervised
 - preserve class separability

Mathematical Foundations

- We need:
 - Mean $\rightarrow$ center data
 - Variance $\rightarrow$ measure spread
 - Covariance $\rightarrow$ measure relationships
 - Eigenvectors $\rightarrow$ directions
 - Eigenvalues $\rightarrow$ importance

Mean and Centering

- Mean

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

- Centering

$$x_i \leftarrow x_i - \mu$$

- PCA requires data centered at the origin

Variance

- Variance: $\text{Var}(x) = \frac{1}{n} \sum (x_i - \mu)^2$
- Large variance $\rightarrow$ more information
- Covariance: $\text{cov}(x, y) = \frac{1}{n} \sum (x_i - \mu_x)(y_i - \mu_y)$
 - >0 : move together
 - <0 : move opposite
 - $=0$: independent

Covariance Matrix

- Symmetric
- Captures relationships between features

$$\Sigma = \begin{bmatrix} \text{var}(x_1) & \text{cov}(x_1, x_2) \\ \text{cov}(x_2, x_1) & \text{var}(x_2) \end{bmatrix}$$

Eigenvalues & Eigenvectors

- $A \vec{v} = \lambda \vec{v}$
- $\vec{v}$: eigenvector (direction)
- λ : eigenvalue (importance)

$$\begin{matrix} A & \vec{v} & = & \lambda \vec{v} \\ \begin{pmatrix} 2 & 3 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 3 \\ 2 \end{pmatrix} & = & \begin{pmatrix} 12 \\ 8 \end{pmatrix} & = & 4 \begin{pmatrix} 3 \\ 2 \end{pmatrix} \end{matrix}$$

In PCA:

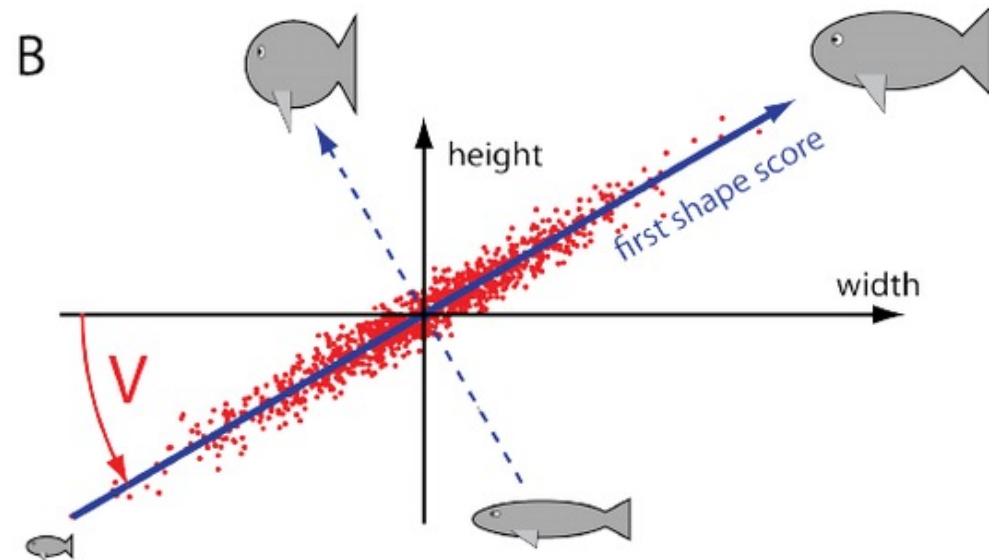
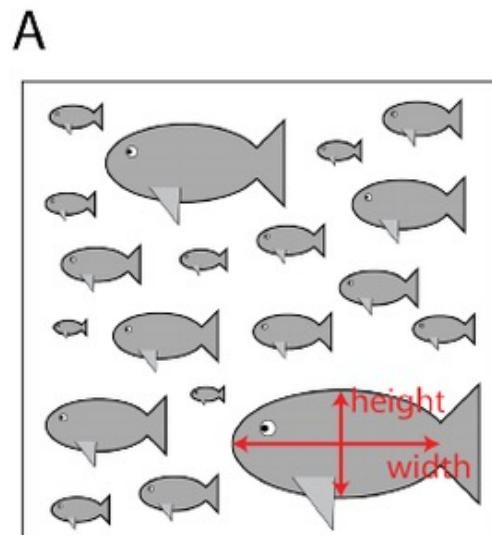
Eigenvectors = **directions of data spread**

Eigenvalues = **how much variance in that direction**

PCA is doing this: Try many directions, find the one where: data spreads the most

PCA Intuition

- Goal:
 - Find direction with maximum variance
- Key idea:
 - Project data onto new axes



PCA Algorithm

Algorithm:

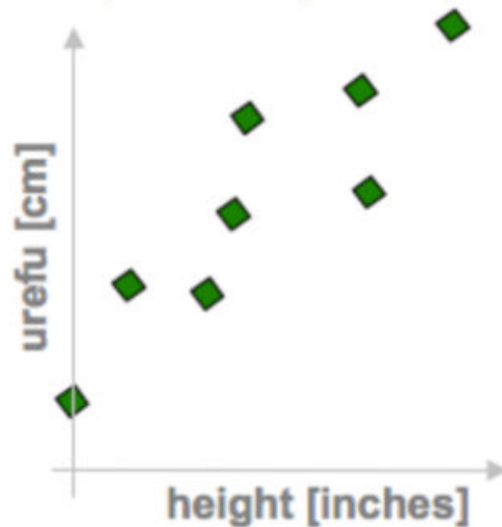
1. Center the data
2. Compute covariance matrix
3. Compute eigenvalues and eigenvectors
4. Sort eigenvectors by eigenvalues
5. Select top k eigenvectors
6. Project data: $Z=XW$

$$\Sigma = \frac{1}{n} X^T X$$

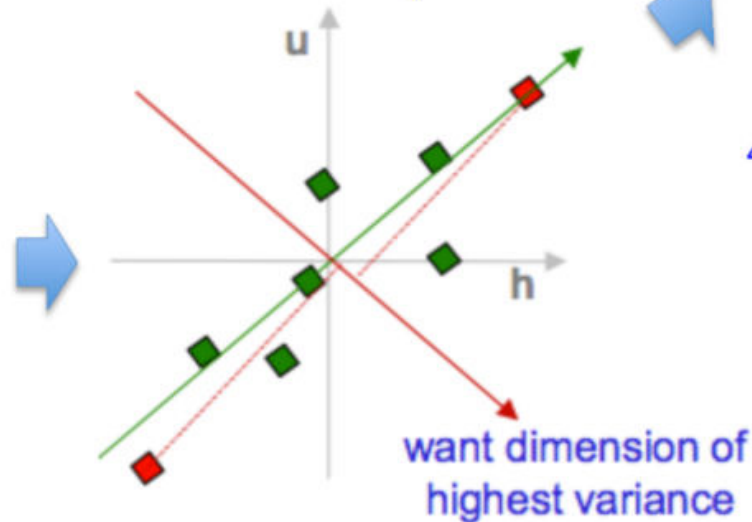
PCA in a nutshell

1. correlated hi-d data

("urefu" means "height" in Swahili)



2. center the points



3. compute covariance matrix

$$\begin{matrix} & h & u \\ h & 2.0 & 0.8 \\ u & 0.8 & 0.6 \end{matrix} \rightarrow \text{cov}(h, u) = \frac{1}{n} \sum_{i=1}^n h_i u_i$$

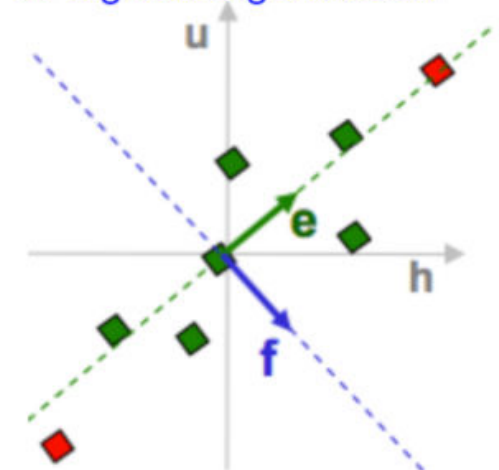
4. eigenvectors + eigenvalues

$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} e_h \\ e_u \end{pmatrix} = \lambda_e \begin{pmatrix} e_h \\ e_u \end{pmatrix}$$

$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} f_h \\ f_u \end{pmatrix} = \lambda_f \begin{pmatrix} f_h \\ f_u \end{pmatrix}$$

`eig(cov(data))`

5. pick m < d eigenvectors w. highest eigenvalues



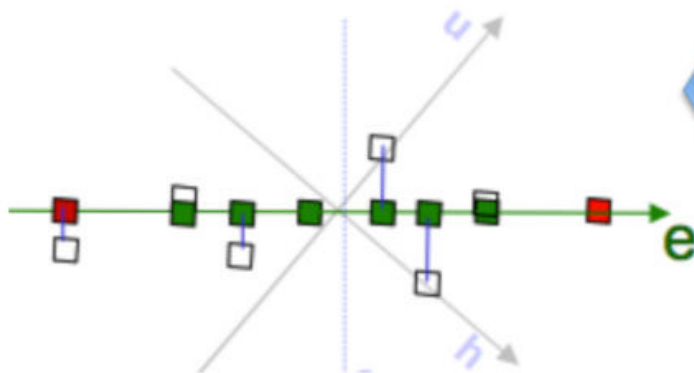
6. project data points to those eigenvectors

$$x'_e = x^T e = \sum_{j=1}^d x_j e_j$$

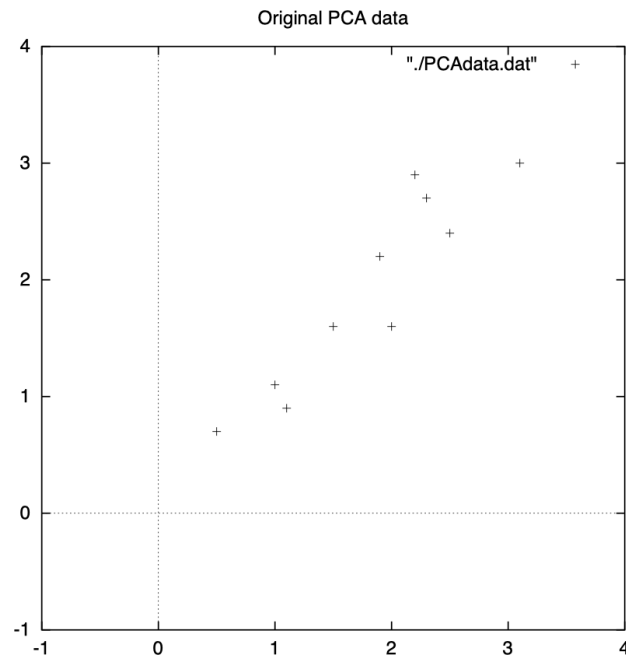
A diagram showing a data point (red circle) being projected onto the principal component axes 'e' and 'f' (dashed lines). The projections are indicated by blue arrows.

Copyright © 2011 Victor Lavrenko

7. uncorrelated low-d data



Step 1&2 Shift data to have 0 mean



Data =

x	y
2.5	2.4
0.5	0.7
2.2	2.9
1.9	2.2
3.1	3.0
2.3	2.7
2	1.6
1	1.1
1.5	1.6
1.1	0.9

DataAdjust =

x	y
.69	.49
-1.31	-1.21
.39	.99
.09	.29
1.29	1.09
.49	.79
.19	-.31
-.81	-.81
-.31	-.31
-.71	-1.01

Step 3: Covariance Matrix

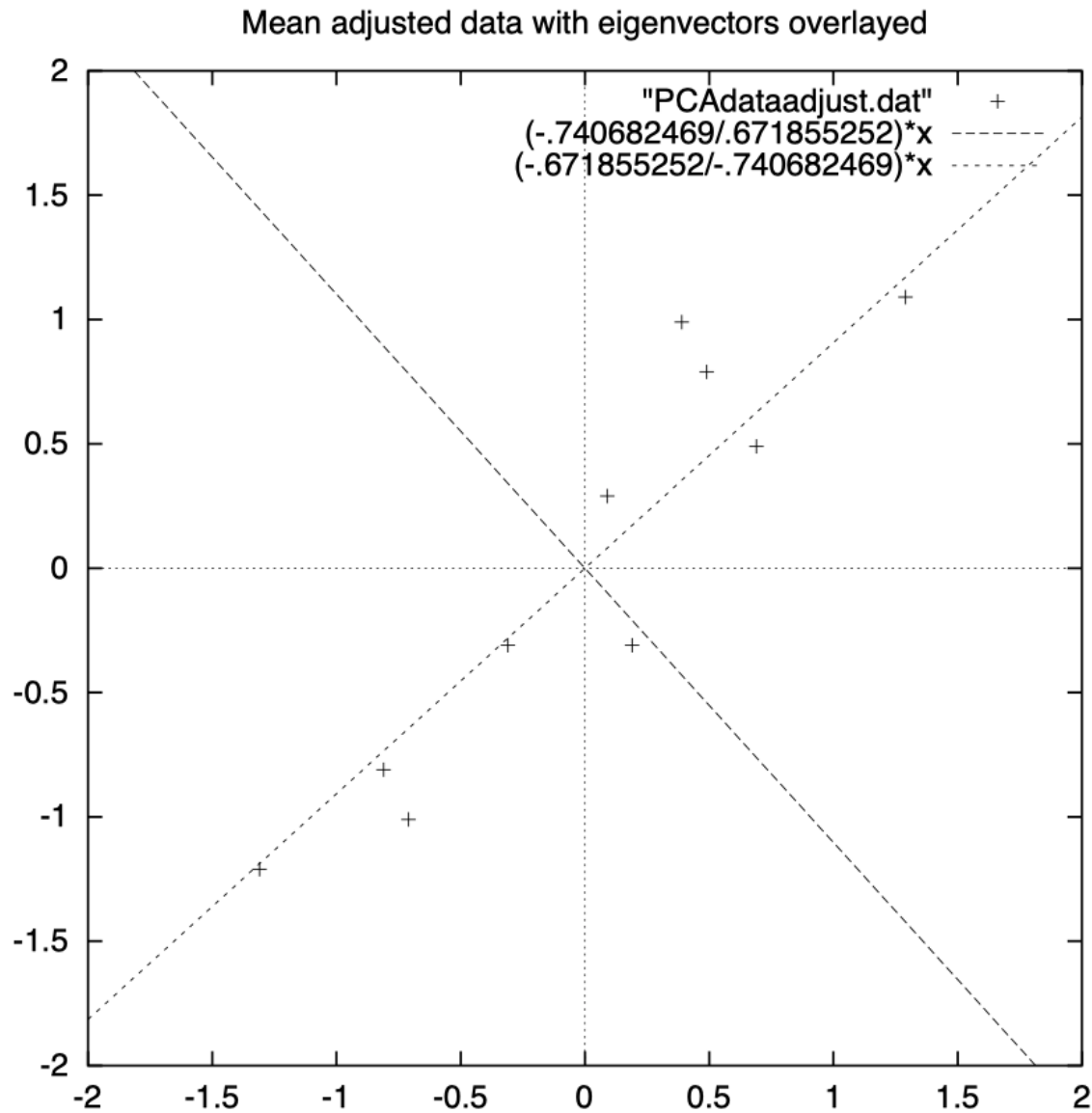
$$cov = \begin{pmatrix} .616555556 & .615444444 \\ .615444444 & .716555556 \end{pmatrix}$$

Step 4: eigenvalues and eigenvectors

$$\text{eigenvalues} = \begin{pmatrix} .0490833989 \\ 1.28402771 \end{pmatrix}$$

$$\text{eigenvectors} = \begin{pmatrix} -.735178656 & -.677873399 \\ .677873399 & -.735178656 \end{pmatrix}$$

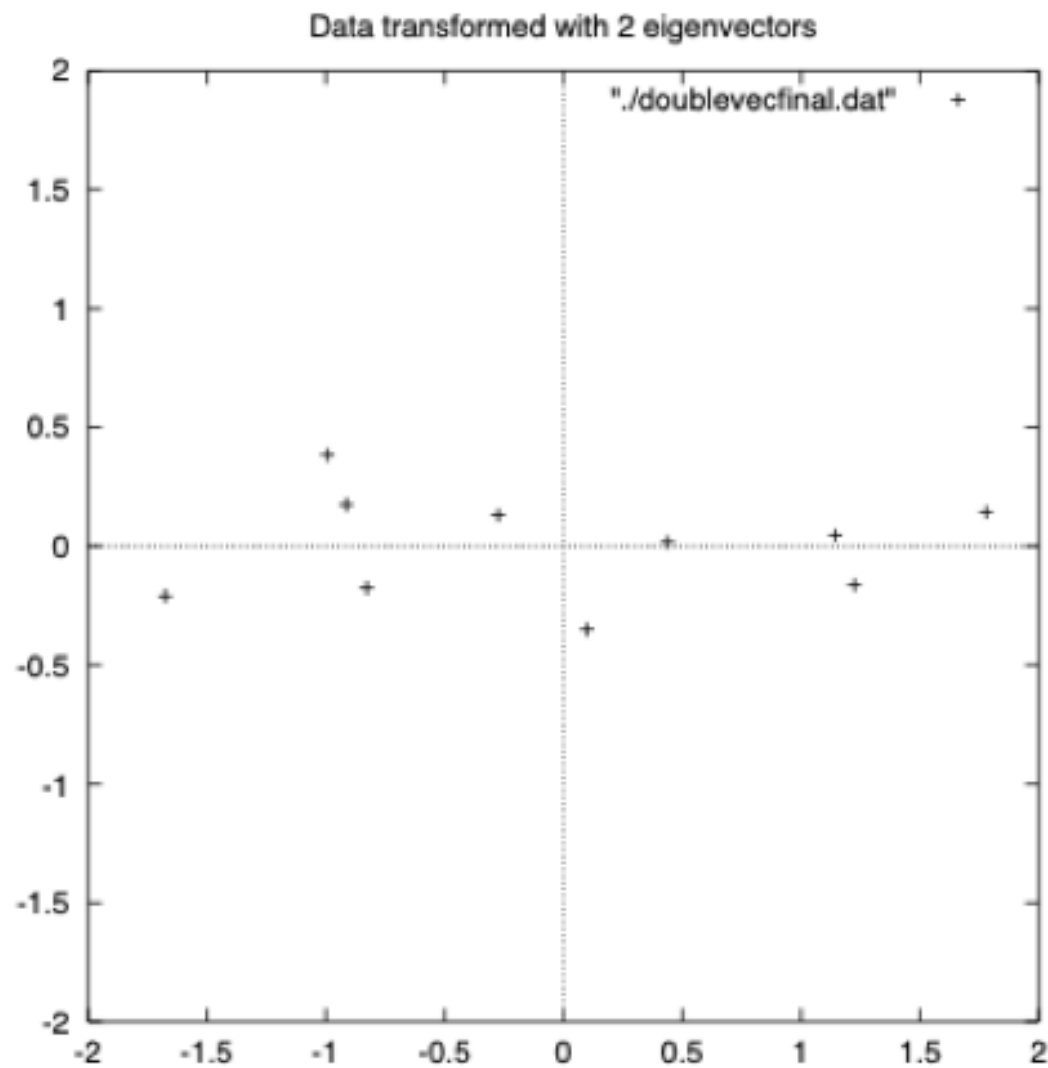
Step 4 result displayed on data



Step 5&6

- Step 5: Select the top K eigenvectors
 - Sort all the eigenvectors in descending order of the corresponding eigenvalues:
 - If d=2 eigenvectors are used:
Column Feature Vector = $\begin{pmatrix} -.677873399 & -.735178656 \\ -.735178656 & .677873399 \end{pmatrix}$
 - If d=1 eigenvector is used:
Column Feature Vector = $\begin{pmatrix} -.677873399 \\ -.735178656 \end{pmatrix}$
- Step 6: Projection data:
$$Z = X'v_1$$
- Explained Variance: Explained ratio = $\frac{\lambda_i}{\sum \lambda}$

PCA result



PCA Objective

Objective: $\max_w \text{Var}(w^T x)$ s.t. $\|w\| = 1$

$w^T x_i$ = projection of x_i onto w

For the whole dataset: $\{w^T x_1, w^T x_2, \dots, w^T x_n\}$

Given data is centered at $\mu=0$,

$$\text{Var}(w^T x) = w^T \Sigma w$$

$$\Sigma = \frac{1}{n} \sum x_i x_i^T \quad \text{covariance between pairs of features}$$

So, optimize: $\max_w w^T \Sigma w$ s.t. $w^T w = 1$

Solving the PCA Objective

To Optimize $\max_w w^T \Sigma w$ s.t. $w^T w = 1$

Use a Lagrange multiplier:

$$\mathcal{L}(w, \lambda) = w^T \Sigma w - \lambda(w^T w - 1)$$

Take derivative with respect to w and set it to 0:

$$\frac{\partial \mathcal{L}}{\partial w} = 2\Sigma w - 2\lambda w = 0$$

So:

$$\Sigma w = \lambda w$$

That is exactly the eigenvalue equation!

$$w^T \Sigma w = w^T (\lambda w) = \lambda w^T w = \lambda$$

λ is the variance

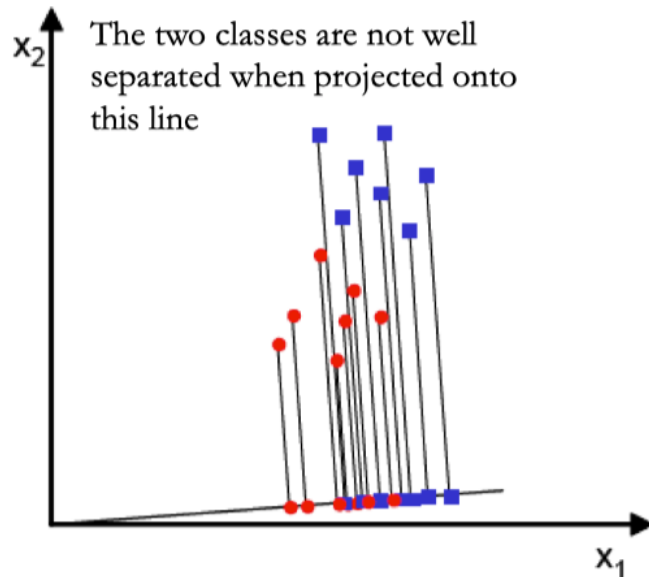
PCA Summary

- Unsupervised
- Maximizes variance
- Ignores labels
- Used when:
 - No labels
 - Visualization
 - Noise reduction

Linear Discriminant Analysis (LDA)

- Goal:
 - Find a projection that best separates classes
- We want:
 - Class means far apart
 - Data within each class tightly clustered
- Maximize distance between class means
- Minimize variance within classes

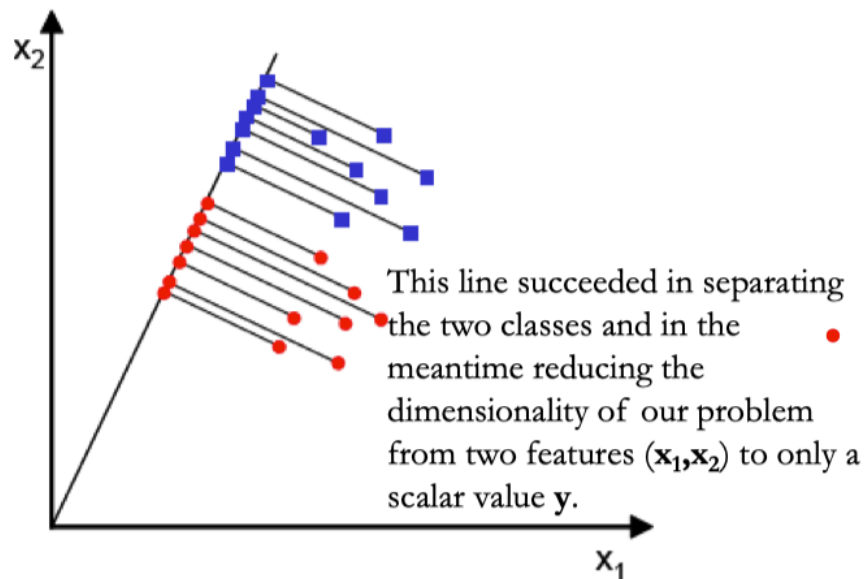
Linear Discriminant Analysis (LDA)



- Assume we have m -dimensional samples $\{\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^N\}$, N_1 of which belong to ω_1 and N_2 belong to ω_2 .
- We seek to obtain a scalar y by projecting the samples $\mathbf{x}$ onto a line ($C-1$ space, $C = 2$).

$$y = \mathbf{w}^T \mathbf{x} \quad \text{where} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \cdot \\ \cdot \\ x_m \end{bmatrix} \quad \text{and} \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \cdot \\ \cdot \\ w_m \end{bmatrix}$$

- where $\mathbf{w}$ is the projection vectors used to project $\mathbf{x}$ to y .



- **Of all the possible lines we would like to select the one that maximizes the separability of the scalars.**

Class Means After Projection

- Original means:

$$\mu_1, \mu_2$$

- Projected means:

$$\tilde{\mu}_1 = w^T \mu_1, \quad \tilde{\mu}_2 = w^T \mu_2$$

- Maximize distance between means:

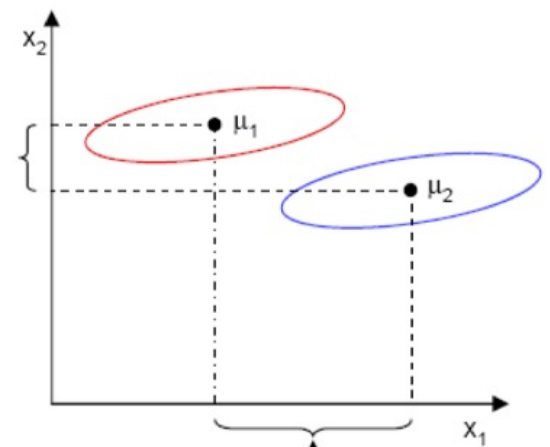
$$|\tilde{\mu}_1 - \tilde{\mu}_2| = |w^T(\mu_1 - \mu_2)|$$

This Is Not Enough:

Ignores spread of data

Classes may still overlap

This axis yields better class separability



This axis has a larger distance between means

Add Variance Consideration

- We want:
 - Large distance between means
 - Small spread within each class
- Within-Class Variance
 - For projected data: $s_i^2 = \sum_{x \in \omega_i} (y - \tilde{\mu}_i)^2$
 - Total for two classes: $s_1^2 + s_2^2$

LDA Objective

- The objective function:

$$J(w) = \frac{(\tilde{\mu}_1 - \tilde{\mu}_2)^2}{s_1^2 + s_2^2}$$

- Next, we compute
 - Between-Class Scatter $(\tilde{\mu}_1 - \tilde{\mu}_2)^2$
 - Within-Class Scatter $s_1^2 + s_2^2$

Between-Class Scatter After Projection

- Projected class means: $\tilde{\mu}_1 = w^T \mu_1, \quad \tilde{\mu}_2 = w^T \mu_2$

- Distance between projected means:

$$(\tilde{\mu}_1 - \tilde{\mu}_2)^2 = (w^T \mu_1 - w^T \mu_2)^2$$

- Factor out w : $= (w^T (\mu_1 - \mu_2))^2$

- Convert to matrix form:

$$= w^T (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T w$$

- Define between-class scatter: $S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$

- Result: $(\tilde{\mu}_1 - \tilde{\mu}_2)^2 = w^T S_B w$

Within-Class Scatter After Projection

- Project data onto direction w : $y = w^T x$
- Variance within class ω_i : $s_i^2 = \sum_{x \in \omega_i} (y - \tilde{\mu}_i)^2 \quad \tilde{\mu}_i = w^T \mu_i$
- Substitute projection: $= \sum_{x \in \omega_i} (w^T x - w^T \mu_i)^2$
- Factor out w : $= \sum_{x \in \omega_i} (w^T (x - \mu_i))^2$
- Convert to Matrix form: $= \sum_{x \in \omega_i} w^T (x - \mu_i)(x - \mu_i)^T w$
- Result:

$$s_i^2 = w^T S_i w \quad S_1 = \sum_{x \in \omega_1} (x - \mu_1)(x - \mu_1)^T$$

Total within-class scatter:

$$s_1^2 + s_2^2 = w^T S_1 w + w^T S_2 w = w^T (S_1 + S_2) w = w^T S_W w$$

LDA Objective Function

- Objective Function: $J(w) = \frac{(\tilde{\mu}_1 - \tilde{\mu}_2)^2}{s_1^2 + s_2^2} = \frac{w^T S_B w}{w^T S_W w}$

- To maximize it, take derivative and set to 0.

$$\frac{d}{dw} J(w) = \frac{d}{dw} \left(\frac{w^T S_B w}{w^T S_W w} \right) = 0$$

- Result: $S_B w = \lambda S_W w \rightarrow S_W^{-1} S_B w = \lambda w$
- The optimal projection direction is the eigenvector of $S_W^{-1} S_B$ with the largest eigenvalue.

Solve the objective function

- To find the maximum of $J(w)$, we differentiate and equate to zero.

$$\frac{d}{dw} J(w) = \frac{d}{dw} \left(\frac{w^T S_B w}{w^T S_W w} \right) = 0$$

$$\Rightarrow (w^T S_W w) \frac{d}{dw} (w^T S_B w) - (w^T S_B w) \frac{d}{dw} (w^T S_W w) = 0$$

$$\Rightarrow (w^T S_W w) 2S_B w - (w^T S_B w) 2S_W w = 0$$

Dividing by $2w^T S_W w$:

$$\Rightarrow \left(\frac{w^T S_W w}{w^T S_W w} \right) S_B w - \left(\frac{w^T S_B w}{w^T S_W w} \right) S_W w = 0$$

$$\Rightarrow S_B w - J(w) S_W w = 0$$

$$\Rightarrow S_W^{-1} S_B w - J(w) w = 0$$

At the optimum:

$$\boxed{\lambda = J(w^*)}$$



$$S_W^{-1} S_B w = \lambda w$$

Closed form solution

Since $S_W^{-1} S_B w = \lambda w$

is the same as: $S_W^{-1} (\mu_1 - \mu_2) (\mu_1 - \mu_2)^T w = \lambda w$

and $(\mu_1 - \mu_2)^T w$ is a scalar:

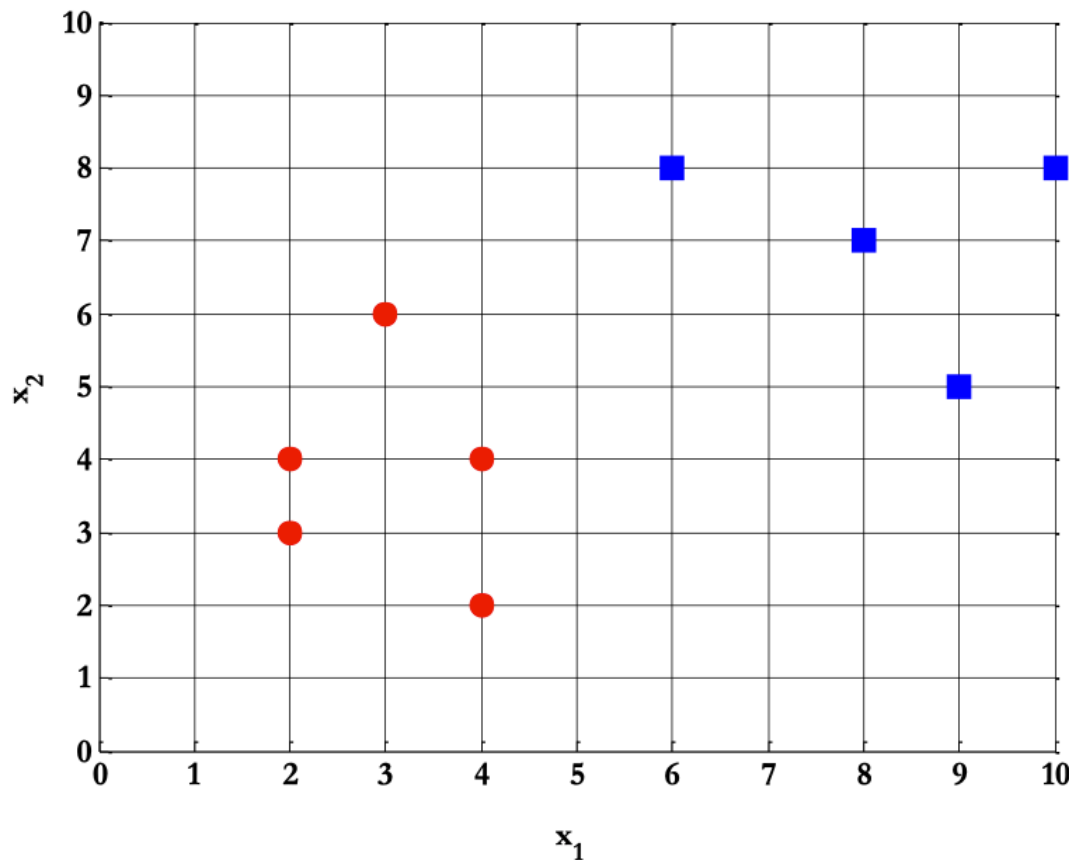
$$S_W^{-1} (\mu_1 - \mu_2) \cdot \underbrace{(\mu_1 - \mu_2)^T w}_{\text{scalar}}$$

the direction :

$$w \propto S_W^{-1} (\mu_1 - \mu_2)$$

An Example

- Compute the Linear Discriminant projection for the following two-dimensional dataset.
 - Samples for class ω_1 : $\mathbf{X}_1=(x_1,x_2)=\{(4,2),(2,4),(2,3),(3,6),(4,4)\}$
 - Sample for class ω_2 : $\mathbf{X}_2=(x_1,x_2)=\{(9,10),(6,8),(9,5),(8,7),(10,8)\}$



An Example: Class mean

- The classes mean are :

$$\mu_1 = \frac{1}{N_1} \sum_{x \in \omega_1} x = \frac{1}{5} \left[\begin{pmatrix} 4 \\ 2 \end{pmatrix} + \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 2 \\ 3 \end{pmatrix} + \begin{pmatrix} 3 \\ 6 \end{pmatrix} + \begin{pmatrix} 4 \\ 4 \end{pmatrix} \right] = \begin{pmatrix} 3 \\ 3.8 \end{pmatrix}$$

$$\mu_2 = \frac{1}{N_2} \sum_{x \in \omega_2} x = \frac{1}{5} \left[\begin{pmatrix} 9 \\ 10 \end{pmatrix} + \begin{pmatrix} 6 \\ 8 \end{pmatrix} + \begin{pmatrix} 9 \\ 5 \end{pmatrix} + \begin{pmatrix} 8 \\ 7 \end{pmatrix} + \begin{pmatrix} 10 \\ 8 \end{pmatrix} \right] = \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix}$$

An Example: Covariance matrix

- Covariance matrix of the first class:

$$\begin{aligned} S_1 &= \sum_{x \in \omega_1} (x - \mu_1)(x - \mu_1)^T = \left[\begin{pmatrix} 4 \\ 2 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^2 + \left[\begin{pmatrix} 2 \\ 4 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^2 \\ &\quad + \left[\begin{pmatrix} 2 \\ 3 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^2 + \left[\begin{pmatrix} 3 \\ 6 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^2 + \left[\begin{pmatrix} 4 \\ 4 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^2 \\ &= \begin{pmatrix} 1 & -0.25 \\ -0.25 & 2.2 \end{pmatrix} \end{aligned}$$

An Example: Covariance matrix

- Covariance matrix of the second class:

$$\begin{aligned} S_2 &= \sum_{x \in \omega_2} (x - \mu_2)(x - \mu_2)^T = \left[\begin{pmatrix} 9 \\ 10 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^2 + \left[\begin{pmatrix} 6 \\ 8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^2 \\ &\quad + \left[\begin{pmatrix} 9 \\ 5 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^2 + \left[\begin{pmatrix} 8 \\ 7 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^2 + \left[\begin{pmatrix} 10 \\ 8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^2 \\ &= \begin{pmatrix} 2.3 & -0.05 \\ -0.05 & 3.3 \end{pmatrix} \end{aligned}$$

An Example: Scatter matrix

- Within-class scatter matrix:

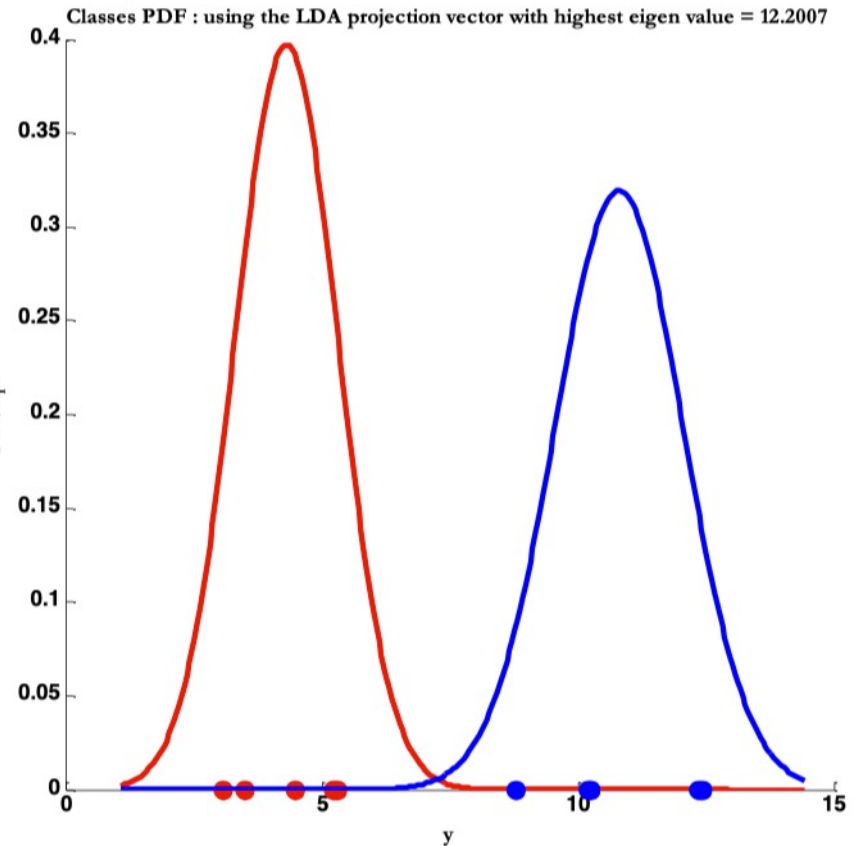
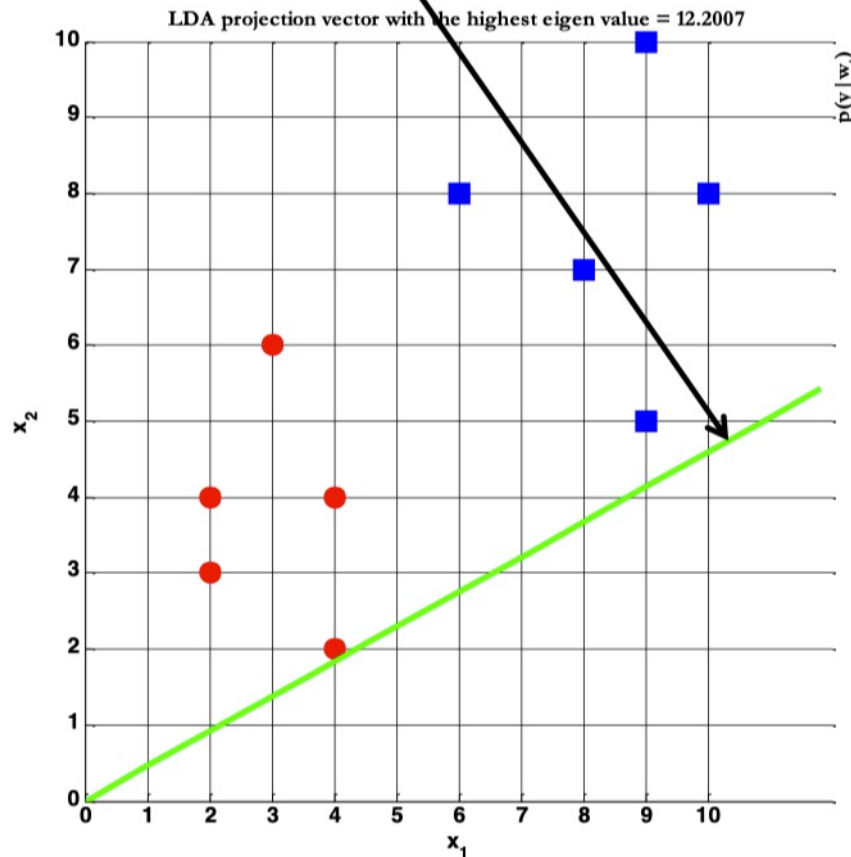
$$\begin{aligned} S_w = S_1 + S_2 &= \begin{pmatrix} 1 & -0.25 \\ -0.25 & 2.2 \end{pmatrix} + \begin{pmatrix} 2.3 & -0.05 \\ -0.05 & 3.3 \end{pmatrix} \\ &= \begin{pmatrix} 3.3 & -0.3 \\ -0.3 & 5.5 \end{pmatrix} \end{aligned}$$

Compute Fisher Linear Discriminant

$$\begin{aligned}w^* &= S_W^{-1}(\mu_1 - \mu_2) = \begin{pmatrix} 3.3 & -0.3 \\ -0.3 & 5.5 \end{pmatrix}^{-1} \left[\begin{pmatrix} 3 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right] \\&= \begin{pmatrix} 0.3045 & 0.0166 \\ 0.0166 & 0.1827 \end{pmatrix} \begin{pmatrix} -5.4 \\ -3.8 \end{pmatrix} \\&= \begin{pmatrix} 0.9088 \\ 0.4173 \end{pmatrix}\end{aligned}$$

Project data with w

The projection vector corresponding to the **highest** eigen value



Using this vector leads to **good separability** between the two classes

An Example: between class scatter matrix

- Between-class scatter matrix:

$$\begin{aligned} S_B &= (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T \\ &= \left[\begin{pmatrix} 3 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right] \left[\begin{pmatrix} 3 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T \\ &= \begin{pmatrix} -5.4 \\ -3.8 \end{pmatrix} \begin{pmatrix} -5.4 & -3.8 \end{pmatrix} \\ &= \begin{pmatrix} 29.16 & 20.52 \\ 20.52 & 14.44 \end{pmatrix} \end{aligned}$$

An Example: solved as a generalized eigenvalue problem

$$S_W^{-1}S_B w = \lambda w$$

$$\Rightarrow |S_W^{-1}S_B - \lambda I| = 0$$

$$\Rightarrow \left| \begin{pmatrix} 3.3 & -0.3 \\ -0.3 & 5.5 \end{pmatrix}^{-1} \begin{pmatrix} 29.16 & 20.52 \\ 20.52 & 14.44 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right| = 0$$

$$\Rightarrow \left| \begin{pmatrix} 0.3045 & 0.0166 \\ 0.0166 & 0.1827 \end{pmatrix} \begin{pmatrix} 29.16 & 20.52 \\ 20.52 & 14.44 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right| = 0$$

$$\Rightarrow \left| \begin{pmatrix} 9.2213 - \lambda & 6.489 \\ 4.2339 & 2.9794 - \lambda \end{pmatrix} \right|$$

$$= (9.2213 - \lambda)(2.9794 - \lambda) - 6.489 \times 4.2339 = 0$$

$$\Rightarrow \lambda^2 - 12.2007\lambda = 0 \Rightarrow \lambda(\lambda - 12.2007) = 0$$

$$\Rightarrow \lambda_1 = 0, \lambda_2 = 12.2007$$

eigenvalues

$$\begin{pmatrix} 9.2213 & 6.489 \\ 4.2339 & 2.9794 \end{pmatrix} w_1 = \underbrace{0}_{\lambda_1} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}$$

and

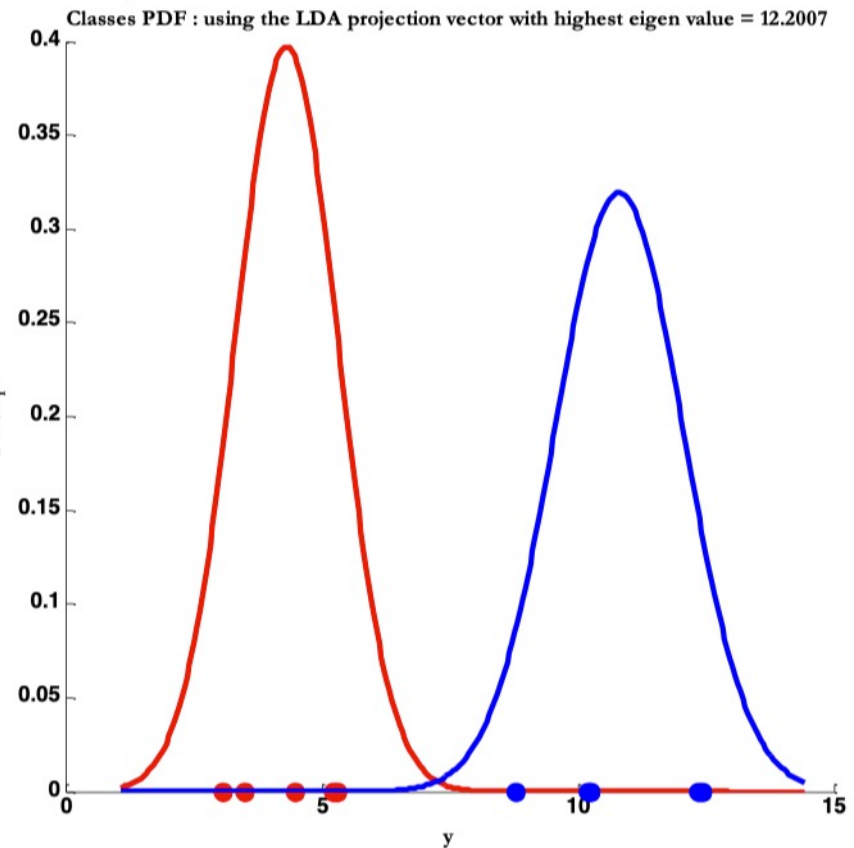
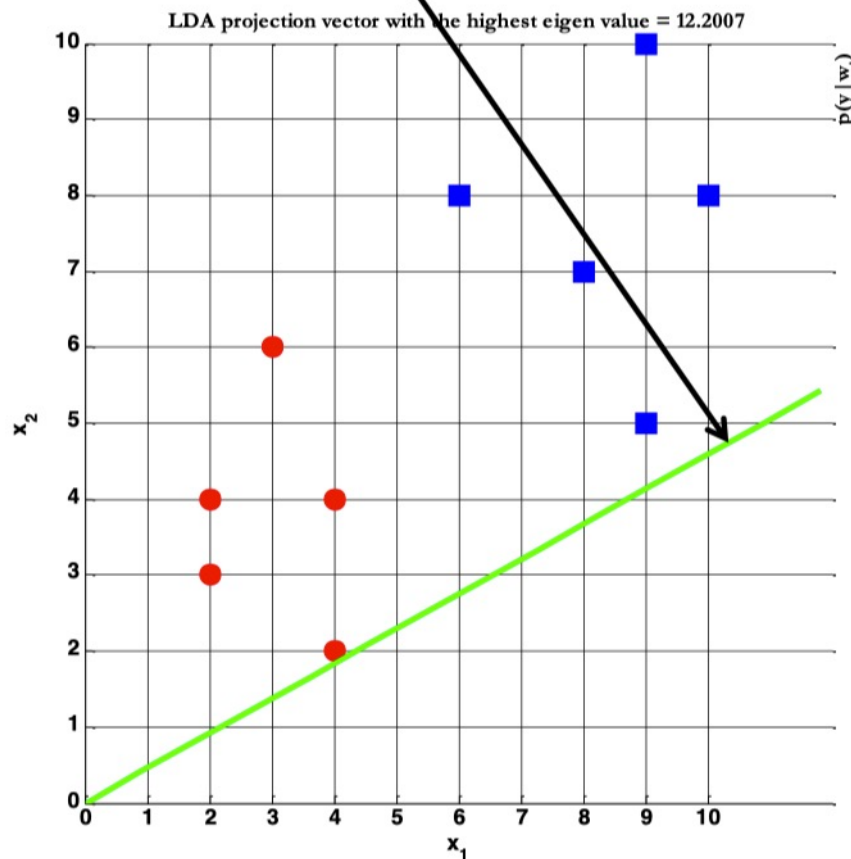
$$\begin{pmatrix} 9.2213 & 6.489 \\ 4.2339 & 2.9794 \end{pmatrix} w_2 = \underbrace{12.2007}_{\lambda_2} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}$$

Thus;

$$w_1 = \begin{pmatrix} -0.5755 \\ 0.8178 \end{pmatrix} \quad \text{and} \quad w_2 = \begin{pmatrix} 0.9088 \\ 0.4173 \end{pmatrix} = w^*$$

Project data with w

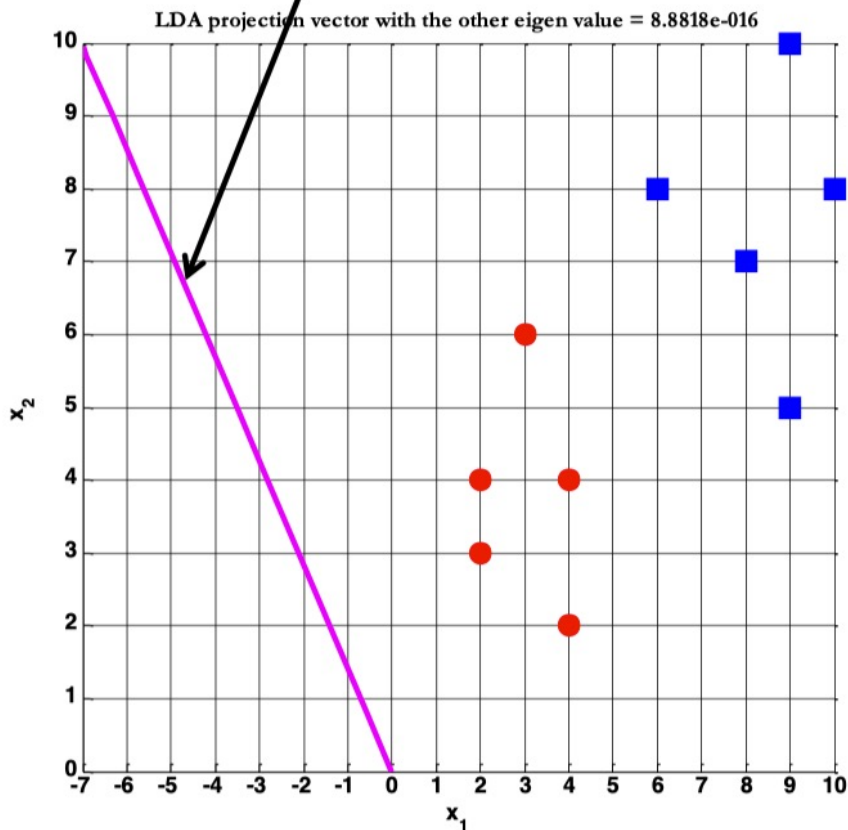
The projection vector corresponding to the **highest** eigen value



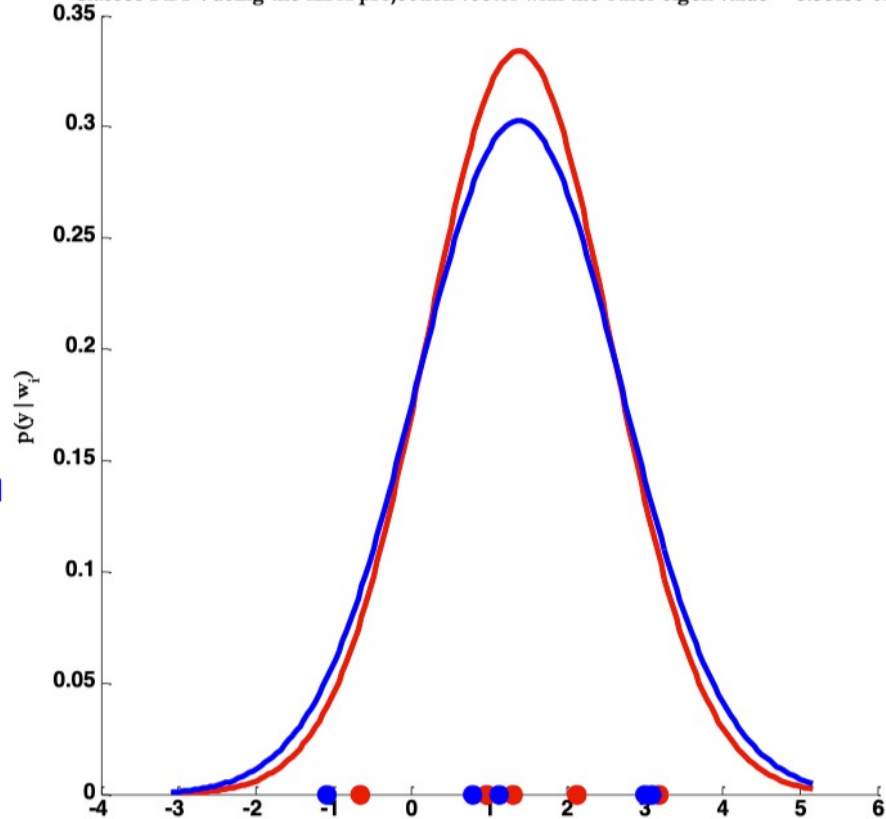
Using this vector leads to **good separability** between the two classes

Project data with w

The projection vector corresponding to the **smallest** eigen value

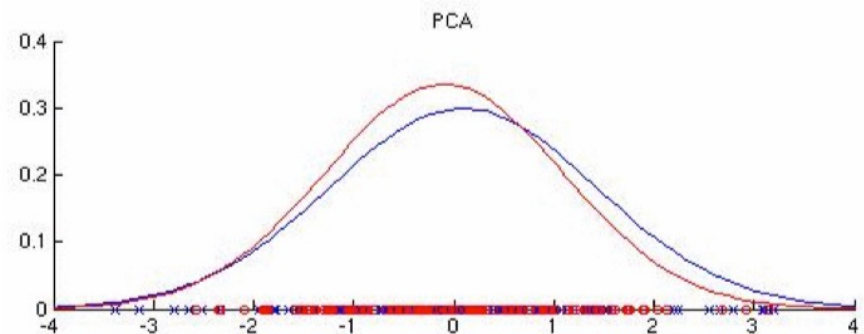
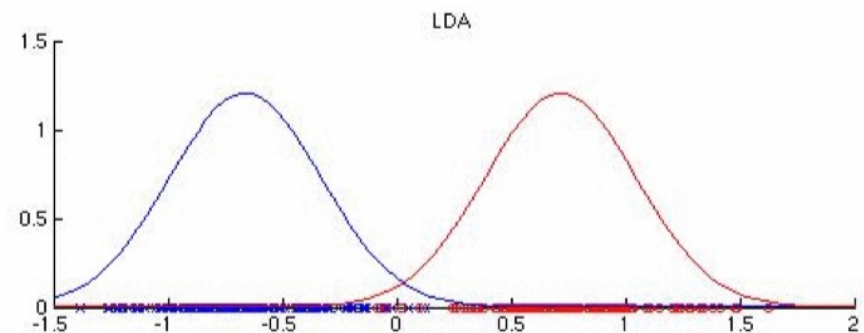
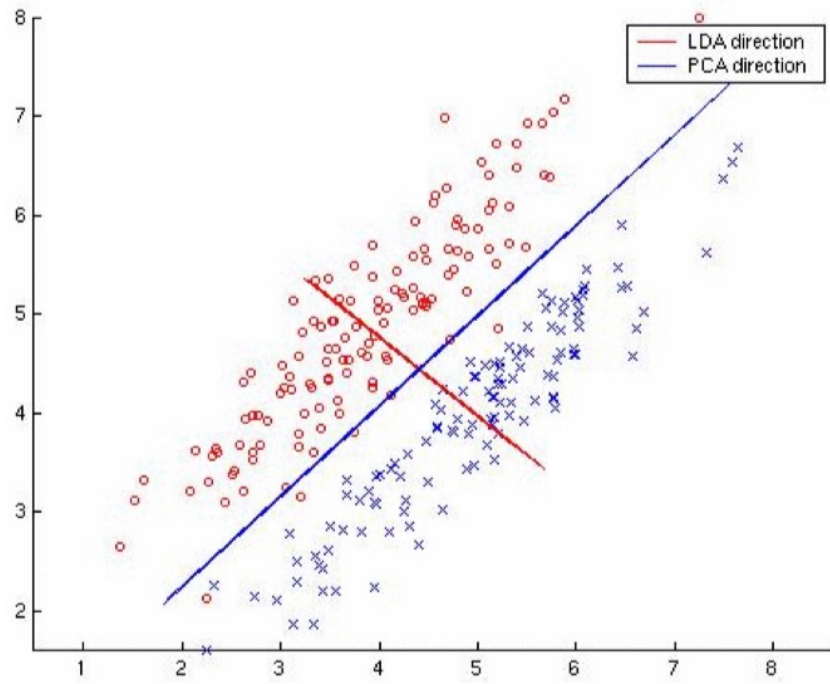


Classes PDF : using the LDA projection vector with the other eigen value = $8.8818\text{e-}016$



Using this vector leads to **bad separability** between the two classes

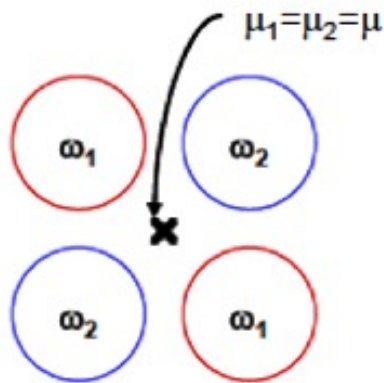
PCA vs LDA



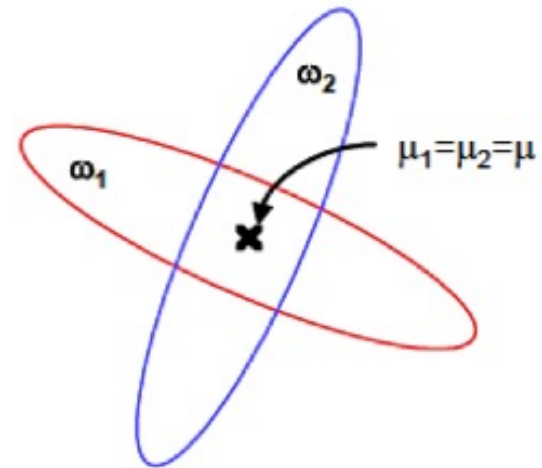
Limitations of LDA

- LDA assumes each class is Gaussian and “simple” (elliptical, unimodal)
- When this is not true $\rightarrow$ LDA can fail

multimodal distributions



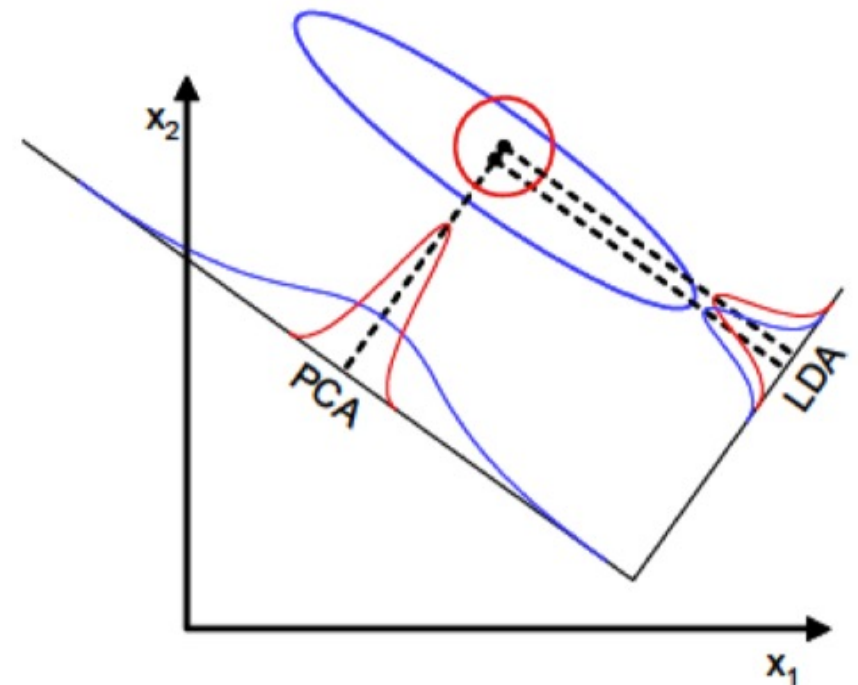
differences in covariance only



nonlinear boundaries

Limitations of LDA

- LDA relies on differences in class means
- If classes have similar means but different covariance structures, LDA may fail
- In such cases, variance-based methods (e.g., PCA) may capture useful structure



Why Do We Need t-SNE?

- Limitations of PCA / LDA:
 - Linear methods
 - Cannot capture complex shapes
 - May fail when data is nonlinear
- Goal:
 - Visualize high-dimensional data in 2D
- Idea:
 - Preserve **local structure (neighbors)** instead of global distances

Core Idea of t-SNE

- For each point:
 - Identify its **nearest neighbors** in high dimension
- In 2D:
 - Place points so that **neighbors stay close**

Points that are close remain close

How t-SNE Works (High-Level)

1. Measure similarity between points in high dimension

p_{ij} = similarity in original space

2. Measure similarity in low dimension

q_{ij} = similarity in 2D

3. Optimize positions to match them

$\min \text{KL}(P \parallel Q)$

Preserve **neighborhood structure**

What t-SNE Preserves (and What It Doesn't)

- Preserves:
 - Local neighborhoods
 - Cluster structure
- Does NOT preserve:
 - Global distances
 - Cluster sizes
 - Relative spacing between clusters

Good for visualization, not for measuring distances

PCA vs LDA vs t-SNE

Method

Goal

PCA

maximize variance

LDA

maximize class separation

t-SNE

preserve neighbors

- When to use t-SNE:
 - Visualizing clusters
 - Exploring high-dimensional data

